

What we're going to cover:

- OOP

Object-Oriented Programming (OOP)

برنامه‌نویسی شیء‌گرا یک روش برنامه‌نویسی است که کد را حول محور اشیاء و کلاس‌ها سازمان‌دهی می‌کند، نه صرفاً بر اساس توابع و منطق

این رویکرد بر ایجاد کدی با قابلیت استفاده مجدد، ماژولار و مقیاس‌پذیر از طریق مدل‌سازی موجودیت‌های دنیای واقعی تمرکز دارد

PYTHON OBJECT-ORIENTED PROGRAMMING CONCEPTS (4-13)



Encapsulation

Hides data, access via methods only



Inheritance

Child class inherits parent features



Polymorphism

Same method, different behavior



Abstraction

Hides complexity using abstract classes



self & cls

Instance vs Class context



Class vs Instance Variables

Shared vs unique attributes



Dunder (Magic) Methods

Customize built-in operations



Operator Overloading

Redefine operators for objects



Composition (HAS-A)

Combine classes modularly

Why Use OOP in Python?

- از طریق وراثت و ترکیب، قابلیت استفاده مجدد از کد را ارتقا می‌دهد

- ماژولار بودن را افزایش می‌دهد — هر کلاس مسئولیت مشخصی دارد
- با جداسازی عملکردها، نگهداری و عیب‌یابی را ساده‌تر می‌کند
- امکان کپسوله‌سازی داده‌ها را فراهم می‌کند و از اطلاعات حساس محافظت می‌نماید
- از چندریختی پشتیبانی می‌کند و امکان رفتار منعطف و پویای کد را فراهم می‌سازد

1. Class and Object

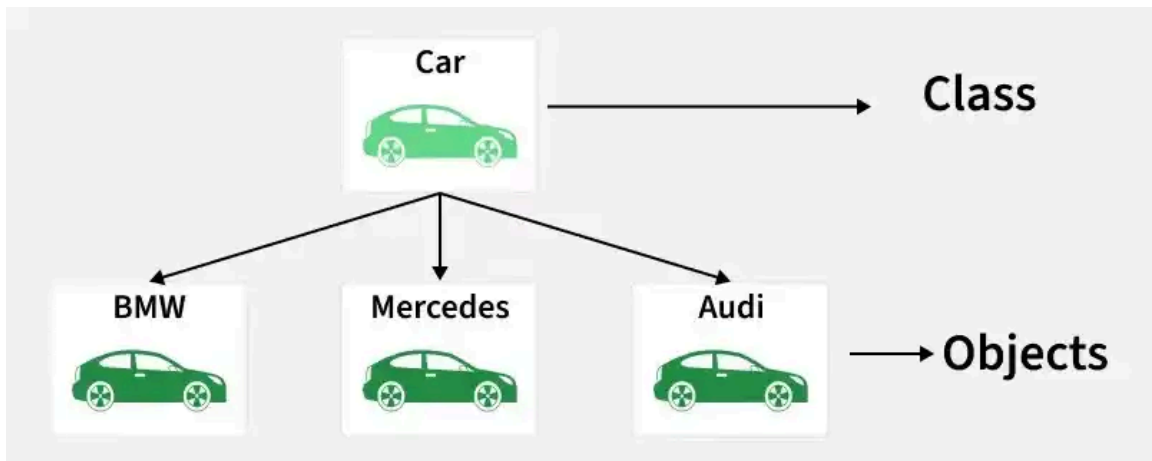
یک کلاس طرح اولیه یا قالبی است که ساختار و رفتار (ویژگی‌ها و متدها) اشیاء را تعیین می‌کند

یک شیء، نمونه‌ای از یک کلاس است که داده‌های واقعی را در خود نگه می‌دارد.

از کلاس‌ها برای نمایش موجودیت‌های دنیای واقعی

مانند خودروها، کاربران یا حساب‌های بانکی در کد استفاده می‌شود

اشیاء ایجادشده از این کلاس‌ها، ضمن داشتن ساختار و عملکرد یکسان، حاوی داده‌های منحصر به فردی هستند



```

In [3]: # define a class
class Bank:
    pass

# Creating object from class
account1 = Bank() # account1 is an object or instance
account2 = Bank()

# Each object has a feature, like name
account1.name = 'Samael' # name is attribute or property
account2.name = 'Tom'
  
```

```
In [4]: class Parrot:

    # class attribute
    name = ""
    age = 0

    # create parrot1 object
    parrot1 = Parrot()
    parrot1.name = "Blu"
    parrot1.age = 10

    # create another object parrot2
    parrot2 = Parrot()
    parrot2.name = "Woo"
    parrot2.age = 15

    # access attributes
    print(f"{parrot1.name} is {parrot1.age} years old")
    print(f"{parrot2.name} is {parrot2.age} years old")
```

Blu is 10 years old

Woo is 15 years old

`__init__`

یک متد (سازنده) ویژه است که هنگام ایجاد یک شیء، ویژگی‌های آن را مقداردهی اولیه می‌کند

`self` پارامتر

به نمونه‌ای که در حال ایجاد شدن است، اشاره دارد

به مثال زیر توجه کنید:

```
In [5]: class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

ایجاد نمونه از اشیاء

پس از تعریف یک کلاس، می‌توانید از آن شیء (نمونه) ایجاد کنید

بیایید یک شیء به نام `person1` از کلاس `Person` بسازیم

```
In [6]: person1 = Person("Alice", 30)
```

دسترسی به ویژگی‌ها و متدها

می‌توانید با استفاده از نشانه‌گذاری نقطه‌ای (dot notation)

به ویژگی‌ها و متدهای یک شیء دسترسی پیدا کنید

برای مثال، جهت دسترسی به ویژگی name در person1

```
In [7]: print(person1.name)
```

Alice

```
In [18]: class Customer:
    def __init__(self, name, account_type):
        """
        Default constructor called automatically when a new object is created.
        Initializes instance variables and can perform setup logic.
        """
        self.name = name
        self.account_type = account_type
        print(f"New customer created: {self.name}, {self.account_type} account")

    c1 = Customer("Alice", "Premium")
    c2 = Customer("Bob", "Normal")
```

New customer created: Alice, Premium account

New customer created: Bob, Normal account

```
In [13]: class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner      # Instance variable for account holder's name
        self.balance = balance  # Instance variable for account balance

    def deposit(self, amount):
        self.balance += amount
        print(f"Deposited {amount}. New balance: {self.balance}")

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            print(f"Withdrew {amount}. Remaining balance: {self.balance}")
        else:
            print("Insufficient balance!")

    # --- Usage ---
    acc1 = BankAccount("Alice", 5000)
    acc2 = BankAccount("Bob", 2000)

    acc1.deposit(1500)
    acc2.withdraw(1000)
```

Deposited 1500. New balance: 6500

Withdrew 1000. Remaining balance: 1000

```
In [20]: class Area:
    pi = 3.14
    def __init__(self, radius):
        self.radius = radius

    def area(self):
```

```
        return self.radius * self.radius * Area.pi    # we can type self.pi or Area.pi

c1 = Area(10)
print(c1.area())
```

314.0

```
In [19]: class Bank:
        bank_name = "National Bank" # Class variable (shared by all instances)

        def __init__(self, account_holder):
            self.account_holder = account_holder # Instance variable (unique to each object)

        def welcome(self):
            print(f"Welcome {self.account_holder} to {Bank.bank_name}!")

# --- Usage examples ---
# Creating two objects (each with its own instance attribute)
customer1 = Bank("Alice")
customer2 = Bank("Bob")
customer1.welcome() # Output: Welcome Alice to National Bank!
customer2.welcome() # Output: Welcome Bob to National Bank!

# Changing the class variable affects all instances
Bank.bank_name = "Global Trust Bank"
customer1.welcome() # Output: Welcome Alice to Global Trust Bank!
customer2.welcome() # Output: Welcome Bob to Global Trust Bank!
```

Welcome Alice to National Bank!
Welcome Bob to National Bank!
Welcome Alice to Global Trust Bank!
Welcome Bob to Global Trust Bank!